

THE EFFICIENT-AI RESEARCH TRACK

The Next 30 Days — Full Study Guide

From "my network has a squash" to "my network reads digits, generates text,
and I made it smaller and measured the cost."

WHAT THIS BOOK IS

A complete, self-contained textbook for your next four weeks. Every concept is taught in full — small numbers first, hand-traces, runnable code, exercises *with answers*. If you run out of AI tokens, this book alone carries you through. Your coach becomes optional; the learning doesn't stop.

Companion to: your Stage 0-1 Study Guide and your live codebase (ai_1_relu.py, ai_2_leaky.py, ai_3_learns.py, ai_v0_readable.py)

How to study from this book (without me)

Our sessions follow a rhythm. Reproduce it solo:

1. **Read one section** (they're sized for one evening session, 10 PM-12 AM style).
2. **Hand-trace before code.** Every section has a trace with small numbers. Do it on paper with a pen. This is non-negotiable — it's the step that made the squash and the nudge finally click for you.
3. **Type the code yourself** into your codebase. Never copy-paste. Typos you fix are lessons.
4. **Do the Try-it.** Answers are at the end of each week — genuinely attempt first.
5. **Write one research-log line:** what worked, what surprised you, what's still foggy.
6. **Commit.** Every session ends with a git commit. Concept → piece → commit.

THE ONE RULE

If a section feels foggy, do NOT push forward. Re-do the hand-trace with different numbers you invent. Fog compounds; clarity compounds too. Depth beats speed on the research track.

Your codebase grows in order: `ai_4_loss.py` → `ai_5_trace.py` → `ai_6_learns_deep.py` → `ai_7_data.py` → `ai_8_digits.py` → `ai_9_measure.py` → `ai_10_text.py` . One story told in files.

WEEK 1 — Closing the Learning Loop

Goal: your 2-layer squash network learns on its own. You already know the neuron, the layer, the squash, and the 1-weight nudge. This week connects them into full backpropagation — slowly, your style.

1.1 The wrongness meter (loss functions)

So far our error was `target - guess`. Fine for one machine — but it has a flaw. Say your network makes two guesses and the errors are **+5** and **-5**. Add them: 0. The meter says "perfect!" while both guesses are badly wrong. Positive and negative errors cancel and hide each other.

The fix is old and simple: **square each error before adding**. Squaring makes everything positive (no more cancelling) and punishes big misses extra hard (an error of 3 costs 9; an error of 10 costs 100 — big mistakes dominate, so the model fixes them first).

```
guesses: [7, 1]      targets: [2, 6]
errors : [7-2, 1-6] = [5, -5]
naive sum  : 5 + (-5) = 0      <- lies to us: "perfect"
squared sum : 25 + 25 = 50     <- tells the truth: "very wrong"
mean squared error (MSE) = 50 / 2 = 25
```

That average-of-squared-errors is called **MSE (mean squared error)** — the most common wrongness meter, and the loss you already met in your math guide's "loss landscape" picture. One number that summarizes "how wrong is the whole model right now." Training = pushing that one number down.

```
# ai_4_loss.py -- the wrongness meter
import numpy as np

def mse(guesses, targets):
    errors = guesses - targets
    return np.mean(errors ** 2)

print(mse(np.array([7., 1.]), np.array([2., 6.]))) # 25.0
```

WHY IT MATTERS

Everything the model learns, it learns by lowering this single number. Choose a bad meter and the model diligently learns the wrong thing. (Failure mode: the naive sum above. Measure: loss should *fall* over training rounds — if it doesn't, something is broken.)

TRY IT 1

Guesses [3, 8, 2], targets [1, 8, 5]. Compute MSE by hand. Then check with your code.

1.2 Which way to nudge? (the slope of the loss)

You know the nudge: `weight += step × input × error`. Where does that formula come from? It's not a trick someone memorized — it falls out of one question: "if I nudge this weight up a hair, does the loss go up or down?" That "which way and how strongly" is exactly the **derivative/slope** from your Stage-1 math — but now the hill is the **loss**, and the ground direction is the **weight**.

Tiny worked example. One machine, one weight: `guess = w × x`, with input $x = 2$, target 6, and w currently 1.

```
w = 1.0      guess = 2      error = guess - target = -4      loss = 16
nudge w up a hair to 1.01:
w = 1.01     guess = 2.02   error = -3.98                  loss = 15.84
loss went DOWN when w went UP -> the slope points down-to-the-right
-> we should push w UP. And the math of the slope says: push it by
   (about) 2 * x * error-with-sign -- which is your nudge formula.
```

So the nudge rule you've been using *is* the slope of the MSE, just written in plain clothes. The general recipe, worth memorizing as a sentence:

THE RECIPE

For each weight, ask: "how much does the loss change if this weight moves a hair?" That's the weight's slope. Then move the weight a small step **against** the slope (downhill). Do it for every weight, repeat thousands of times. That's all training is.

TRY IT 2

Same setup ($x=2$, $\text{target}=6$). Set $w = 4$ (now the guess is 8 — too HIGH). Nudge w up a hair to 4.01 and recompute the loss. Did it rise or fall? So which direction should w move? Check that your nudge formula agrees.

1.3 Blame across two layers (backpropagation, slowly)

Now the week's summit. You've seen this once with leverage; here it is drawn out fully. Two machines in a chain, single numbers:

```
input 2 --[w1 = 1]--> middle m --[w2 = 2]--> guess      target = 12

FORWARD (the guess):   m = 2 × 1 = 2          guess = 2 × 2 = 4
COMPARE:               error = 4 - 12 = -8      (too low by 8)
```

Both weights share the blame. The rule for splitting it is the **leverage** idea, and it has a mechanical recipe — *blame flows backward, multiplying by whatever it passes through*:

```

BACKWARD (the blame):
blame at guess      = -8          (that's just the error)
blame reaching w2    = -8 x m = -8 x 2 = -16      (w2's lever = what flowed INTO it)
blame passing w2 to m = -8 x w2 = -8 x 2 = -16      (to go past w2, multiply by w2)
blame reaching w1    = -16 x input = -16 x 2 = -32 (w1's lever = what flowed into IT)

NUDGE (step 0.01, move against the blame):
w2: 2 - 0.01 x (-16) = 2.16
w1: 1 - 0.01 x (-32) = 1.32

```

Re-run the forward pass with the new weights: $m = 2 \times 1.32 = 2.64$, $\text{guess} = 2.64 \times 2.16 = \mathbf{5.70}$. Last round it was 4. The target is 12. One nudge and the guess jumped closer — both layers learned, including the buried one. Repeat a few hundred times and it lands on 12.

Notice the two-part pattern in the backward pass — this is the whole algorithm:

- **To blame a weight:** (blame arriving from the right) $\times$ (the value that flowed *into* that weight from the left).
- **To send blame further left past a weight:** (blame arriving) $\times$ (that weight itself).

That's backpropagation. Two multiply rules, applied right-to-left. Everything else is bookkeeping.

Where does the squash fit?

One more piece and the picture is complete. If a squash sits in the middle, blame must pass through it too — multiplied by the squash's **pass-through: 1** where the score was positive (gate open), **0** for normal ReLU where the score was negative (gate shut — the dead-neuron freeze you discovered), or **0.1** for your leaky squash (gate ajar — always trainable). Your ReLU-vs-Leaky experiment was exactly this number in action.

TRY IT 3 (THE BIG ONE)

Same chain, but start $w_1 = 2$, $w_2 = 1$, input 2, target 12. Do the full forward, backward, and one nudge (step 0.01) by hand. Then verify the new guess improved. If you can do this cold, you understand backprop better than most people who use it daily.

1.4 Ship it: the readable deep learner

Now assemble the week into code — your style: rows are neurons, one example at a time, every line traceable. This is `ai_6_learns_deep.py`, the readable version of the XOR file that once confused you:

```

# ai_6_learns_deep.py -- a 2-layer network that LEARNS, readable style
import numpy as np
np.random.seed(7)

def squash(z):      return np.where(z > 0, z, 0.1*z)  # leaky
def pass_through(z): return np.where(z > 0, 1.0, 0.1)  # its gate

# XOR: output 1 only when the two inputs differ
examples = [(0,0,0), (0,1,1), (1,0,1), (1,1,0)]

H = 4                                # 4 hidden neurons
W1 = np.random.randn(H, 2) * 0.8     # each ROW = one hidden neuron
b1 = np.zeros(H)
W2 = np.random.randn(1, H) * 0.8     # final machine reads H outputs
b2 = np.zeros(1)
step = 0.1

for round in range(8001):
    total_loss = 0
    for (a, c), target in examples:
        x = np.array([a, c], dtype=float)

        # ---- FORWARD: the guess ----
        z1 = W1 @ x + b1              # raw scores, layer 1
        h = squash(z1)                # squashed
        guess = (W2 @ h + b2)[0]      # final score

        # ---- COMPARE ----
        error = guess - target
        total_loss += error**2

        # ---- BACKWARD: blame, right to left ----
        blame_out = 2 * error          # from the loss
        blame_w2 = blame_out * h       # x what flowed in
        blame_b2 = blame_out
        blame_h = blame_out * W2[0]    # past W2: x the weight
        blame_z1 = blame_h * pass_through(z1) # through the squash gate
        blame_w1 = np.outer(blame_z1, x) # x what flowed in
        blame_b1 = blame_z1

        # ---- NUDGE: against the blame ----
        W2 -= step * blame_w2; b2 -= step * blame_b2
        W1 -= step * blame_w1; b1 -= step * blame_b1

    if round % 2000 == 0:
        print(f"round {round:>5}  loss {total_loss:.4f}")

print("\nfinal answers:")
for (a, c), target in examples:
    x = np.array([a, c], dtype=float)
    g = (W2 @ squash(W1 @ x + b1) + b2)[0]
    print(f"  {a},{c} -> {g:.2f}    (want {target})")

```

Expected output: loss starts around 1-2, collapses toward 0.00, and the four answers land within a few hundredths of 0, 1, 1, 0. The only genuinely new line is

`np.outer(blame_z1, x)` — it just builds the blame table for W1: each hidden neuron's blame $\times$ each input, laid out in the same rows-as-neurons shape as W1 itself.

EFFICIENCY LENS

Count what the backward pass reused: z_1, h, x — all values *already computed going forward*. Backprop's genius is recycling; the backward pass costs only about 2x the forward pass, no matter how deep the network. Any "cheaper training" idea you ever have must beat that bar. Now you know the bar.

Week 1 exercises

1. Change the squash in `ai_6` to normal ReLU (`np.maximum(0,z)` and gate `1.0/0.0`). Run 5 times with seeds 1-5. How often does it fail to learn XOR vs leaky? (You are re-running your dead-neuron discovery inside a real network.)
2. Change step to 1.0. What happens to the loss? Explain using the learning-rate picture from your Stage-1 guide.
3. Reduce H to 1 hidden neuron. Can it still learn XOR? Why might one bend not be enough?

ANSWERS: TRY-ITS

1: errors $[2, 0, -3] \rightarrow$ squares $[4, 0, 9] \rightarrow$ MSE = $13/3 \approx 4.33$. **2:** $w=4.01 \rightarrow$ guess 8.02 $\rightarrow$ loss rises (from 4.00 to 4.08). Loss rose when w rose, so move w DOWN — and the nudge formula agrees (error is +2, so $w -= \text{step} \cdot x \cdot \text{error}$ pushes w down). **3:** forward: $m=4$, guess=4, error=-8. backward: blame $w_2 = -8 \times 4 = -32$; blame past $w_2 = -8 \times 1 = -8$; blame $w_1 = -8 \times 2 = -16$. nudge: $w_2=1.32$, $w_1=2.16$. new guess: $m=4.32$, guess=5.70 — improved from 4.

ANSWERS: EXERCISES (CHECK AFTER ATTEMPTING)

1: plain ReLU fails on some seeds (neurons die, loss plateaus); leaky recovers on all — same finding as your experiment, now in a full net. **2:** loss explodes/oscillates — steps overshoot the valley. **3:** usually no: XOR needs at least 2 bends to fold the diagonal pattern; 1 hidden neuron gives only one crease.

WEEK 2 — Real Data In

Goal: the same network learns to read handwritten digits. Data stops being toy numbers; probability from Stage 1 becomes practical.

2.1 What a dataset really is (features, labels, and the honest split)

A dataset is just rows of **(input, correct answer)** — exactly like your File-3 examples, only more of them. The input's numbers are called **features**; the correct answer is the **label**.

The one idea that keeps all of machine learning honest: **split your data before training**.

```
all examples (say 1000)
|-- TRAIN set (800): the model learns from these
|-- TEST set (200): locked in a drawer; the model NEVER sees them
Final grade = accuracy on the TEST set only.
```

Why? Remember File 3's proudest moment: it answered **[4,1] → 12** — an input it had *never seen*. That's what proved it learned the rule instead of memorizing the examples. The test set is that moment, industrialized: a standing supply of never-seen questions. Grading on training data is like grading a student on the exact worksheet they practiced — a memorizer scores 100% and learns nothing.

FAILURE MODE: LEAKAGE

If test examples sneak into training (a duplicate row, peeking at test results to tune), your score becomes a lie — usually an impressive lie. Researchers call it **data leakage** and it is the #1 source of "amazing results" that evaporate. If your result looks too good: check for leakage FIRST, celebrate second.

```
# ai_7_data.py -- load real data and split honestly
from sklearn.datasets import load_digits
import numpy as np
np.random.seed(0)

digits = load_digits()           # 1797 handwritten digits, 8x8 pixels
X, y = digits.data, digits.target # X: rows of 64 numbers. y: labels 0-9

order = np.random.permutation(len(X)) # shuffle BEFORE splitting!
X, y = X[order], y[order]
split = int(0.8 * len(X))
X_train, y_train = X[:split], y[:split]
X_test, y_test = X[split:], y[split:]
print(X_train.shape, X_test.shape)    # (1437, 64) (360, 64)
```

Each image is 8x8 = **64 pixels**, flattened into one row of 64 numbers (each 0-16, dark to bright). To your network it's just a longer movie: instead of 2 features (action, comedy), 64 features. Nothing conceptually new.

TRY IT 1

Why shuffle before splitting? Hint: the digits dataset is stored roughly in order 0,0,0...1,1,1...9,9,9. What would the test set contain if you split without shuffling?

2.2 Look at your data (always, before anything)

A researcher never trains on data they haven't looked at. Two minutes of looking prevents weeks of confusion.

```
import matplotlib.pyplot as plt
fig, axs = plt.subplots(1, 8, figsize=(10, 2))
for i, ax in enumerate(axs):
    ax.imshow(X_train[i].reshape(8, 8), cmap="gray_r")
    ax.set_title(int(y_train[i])); ax.axis("off")
plt.savefig("samples.png")

print("pixel range :", X.min(), "to", X.max())    # 0.0 to 16.0
print("pixel mean  :", X.mean().round(2))
print("labels count:", np.bincount(y))           # ~180 of each digit: balanced
```

Three things to check on any dataset, ever: the **range** of the features (here 0-16), whether classes are **balanced** (here yes — about 180 of each digit; if one class dominated, accuracy would mislead you), and a few **actual samples** with your eyes (is a "3" recognizably a 3?).

2.3 Scaling: why big inputs break the nudge

Your nudge multiplies by the input: **blame_{W1} = blame × input**. If inputs are big (16) in some places and tiny (0) in others, some weights get violently shoved while others barely move — training wobbles or explodes. Hand-feel it:

```
same error = 1, same step = 0.1
input = 16 -> nudge = 0.1 × 16 × 1 = 1.6      (a huge lurch)
input = 0.5 -> nudge = 0.1 × 0.5 × 1 = 0.05    (a tiptoe)
One dial spins wildly, another crawls: the descent zigzags.
```

The fix is one line — put every pixel on a 0-to-1 scale:

```
X_train = X_train / 16.0
X_test = X_test / 16.0    # scale test the SAME way -- never differently
```

FAILURE MODE

Scaling train and test differently (or computing the scale from test data) is a subtle leak. Rule: decide every preprocessing step using the training set only, then apply it unchanged to test.

2.4 Ten answers instead of one (multi-class output)

The digit task has ten possible answers (0-9). The change is smaller than it looks: the final layer gets **10 machines instead of 1** — W2 becomes 10 rows — so the network outputs **10 scores**. The prediction is simply *the highest score's index*.

```
output scores: [ 0.1, 0.3, 2.9, 0.2, 0.1, 0.4, 0.2, 8.1, 0.3, 1.2 ]
index:         0   1   2   3   4   5   6   7   8   9
highest = index 7   ->   the network says: "this is a 7"
```

The label needs the same shape. We turn "the answer is 7" into a target of ten numbers — one 1 in position 7, zeros elsewhere (called **one-hot**): `[0,0,0,0,0,0,0,1,0,0]`. Then MSE compares 10 guesses to 10 targets exactly as before. (Real systems use a fancier meter called cross-entropy here — it punishes confident wrongness harder. MSE genuinely works for a first build; we upgrade later, one change at a time.)

2.5 Ship it: your network reads digits

`ai_8_digits.py` = `ai_6` with three edits: input size 2→64, output size 1→10, more hidden neurons (H=32). The learning loop — forward, compare, blame, nudge — is *unchanged*. That's the beauty: the machinery you hand-traced on single numbers scales to reading handwriting without modification.

```
# ai_8_digits.py -- key parts (start from your ai_6!)
H = 32
W1 = np.random.randn(H, 64) * np.sqrt(2/64) # see note below
b1 = np.zeros(H)
W2 = np.random.randn(10, H) * np.sqrt(2/H)
b2 = np.zeros(10)
step = 0.05

def one_hot(label):
    t = np.zeros(10); t[label] = 1.0; return t

for round in range(30): # 30 passes over the data
    for i in range(len(X_train)):
        x, target = X_train[i], one_hot(y_train[i])
        z1 = W1 @ x + b1
        h = squash(z1)
        out = W2 @ h + b2 # 10 scores now
        error = out - target # 10 errors now
        blame_W2 = np.outer(2*error, h) # same two blame rules,
        blame_h = W2.T @ (2*error) # just 10 lanes at once
        blame_z1 = blame_h * pass_through(z1)
        blame_W1 = np.outer(blame_z1, x)
        W2 -= step*blame_W2; b2 -= step*2*error
        W1 -= step*blame_W1; b1 -= step*blame_z1

# honest grade after each pass:
correct = 0
for i in range(len(X_test)):
    scores = W2 @ squash(W1 @ X_test[i] + b1) + b2
    if np.argmax(scores) == y_test[i]: correct += 1
print(f"pass {round:>2} test accuracy {100*correct/len(X_test):.1f}%")
```

Expect roughly: ~20–40% after the first pass, above 90% within 10–20 passes, settling around 93–96%. If you see that: stop and appreciate it. A machine you wrote from scratch — every line hand-traceable — is reading human handwriting.

Note on `np.sqrt(2/64)` : starting weights are scaled DOWN as inputs grow, so layer-1 scores don't start huge (huge scores = saturated squashes = slow start). It's the standard starting-size recipe ("He initialization"). One line, big stability win — and it's exactly the anti-dead-neuron setup we discussed.

TRY IT 2

Print training accuracy alongside test accuracy each pass. Which is higher, and why is a small gap expected and honest?

2.6 Experiment day: one dial at a time

Researcher rule: **change one thing, measure, write it down BEFORE and after**. Two experiments, hypothesis first:

Dial	Values to try	Write your hypothesis first	Measure
Hidden neurons H	4, 16, 32, 64	"More neurons will ___ accuracy and ___ time"	final test acc + seconds/pass
Step (learning rate)	0.005, 0.05, 0.5	"0.5 will ___ because ___"	the accuracy curve's shape

EFFICIENCY LENS

The H experiment is your first real capability-vs-cost curve: going 4→32 buys a lot of accuracy; 32→64 buys little but doubles the cost. Diminishing returns per unit of compute is THE core pattern of efficiency research — the same shape as the scaling laws that govern billion-dollar training runs. You just measured it on a laptop.

Week 2 exercises

1. Build a **confusion table**: a 10×10 grid, rows = true digit, columns = predicted; add 1 at `[true][pred]` for each test example. Which two digits get confused most? Look at a few misclassified images — can YOU tell what they are?

2. Remove scaling (train on raw 0–16 pixels). What happens to the accuracy curve, and which section of this week explains it?
3. Shrink training data to 200 examples (keep the same test set). How far does accuracy fall? Log it — this is your first data-efficiency measurement, and the exact lever your research cares about.

ANSWERS

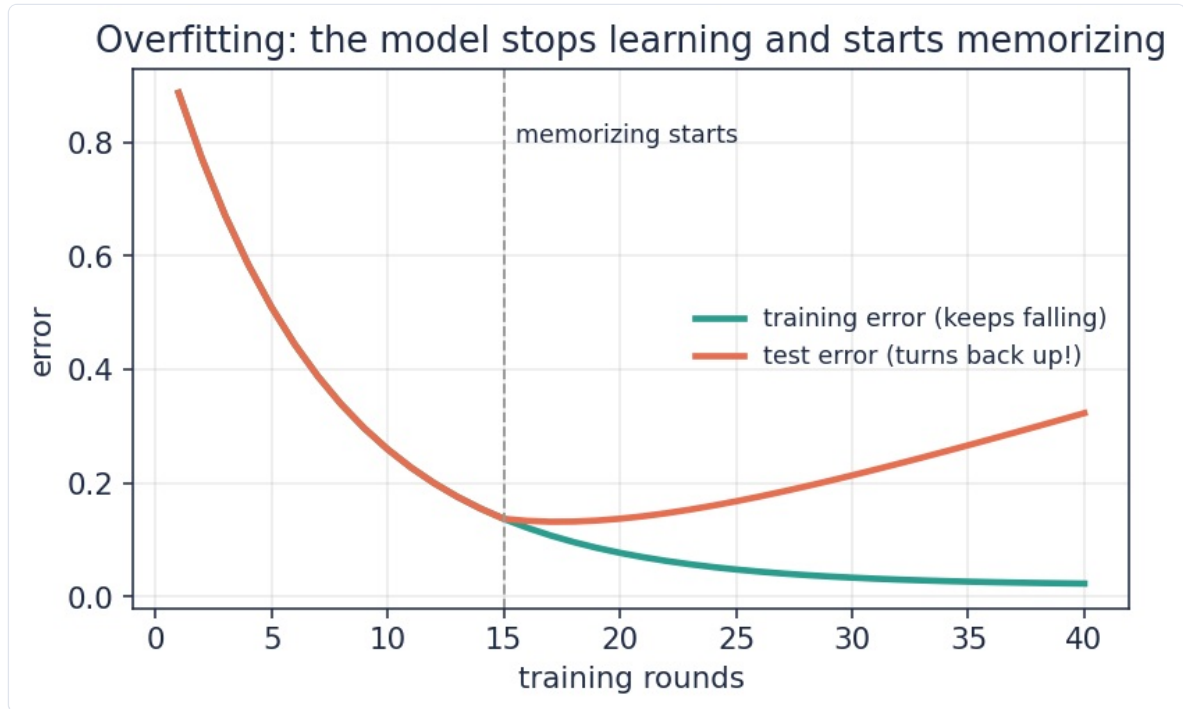
Try-it 1: unshuffled, the test set would be mostly 8s and 9s — the model would be graded only on digits it barely saw. **Try-it 2:** training accuracy is a few points higher; the model has literally seen those examples. A small gap is normal — a huge gap is Week 3's topic. **Ex 1:** commonly 1↔7, 3↔8, or 9↔5 — genuinely similar shapes at 8×8. **Ex 2:** slower, wobblier learning; section 2.3 (uneven nudges). **Ex 3:** typically drops to ~80–88% — less data, less rule-finding; curve is worth keeping for your research log.

WEEK 3 — Make It Solid, Then Make It Cheaper

Goal: first, the failure that fools everyone (overfitting) and its fixes. Then your first two REAL efficiency experiments — pruning and quantization — on your own model. The research begins.

3.1 Overfitting: the memorizer in disguise

Train your digit network for a very long time and something strange happens: training error keeps falling... while **test** error stops falling and creeps back UP.



The signature of overfitting: the two curves separate. The gap = memorization.

What's happening: early on, the network learns the real, general rules of digit-shape ("a 7 has a horizontal bar up top"). But given endless rounds, it starts memorizing quirks of the specific training images ("training image #612 has a smudge in the corner"). Quirks boost the training grade and do nothing — or harm — on fresh data. The gap between the two curves is exactly your File-3 question inverted: *memorizing the worksheet instead of learning the rule*.

WHY / FAILURE / MEASURE

Why it happens: the model has enough capacity to store quirks, and nothing stops it. **Failure mode:** celebrating training accuracy — always report test. **Measure:** track BOTH curves; the moment they separate is the moment learning turned into memorizing.

3.2 The fixes (and why one of them is free money)

Fix 1 — Early stopping. Watch test accuracy each pass; keep a copy of the best-so-far weights; if no improvement for, say, 5 passes, stop and keep the copy. Simple, honest, and note what it also is: **fewer passes = less compute = cheaper training**. The very first efficiency trick is literally "stop when it's done."

```
best_acc, best_weights, patience = 0, None, 0
# inside the training loop, after measuring test accuracy:
if acc > best_acc:
    best_acc = acc
    best_weights = (w1.copy(), b1.copy(), w2.copy(), b2.copy())
    patience = 0
else:
    patience += 1
    if patience >= 5:
        print("stopping early - restoring best"); break
```

Fix 2 — Weight decay (a taste). Each nudge, also shrink every weight a tiny bit: `w *= 0.9999`. Quirk-memorizing tends to need weird, large weights; a constant gentle pull toward zero makes quirks expensive to keep and general rules (supported by many examples, constantly re-strengthened) cheap. One line; often worth a point of test accuracy.

3.3 Measure the cost of your own model

Before making anything cheaper: measure the baseline. Your forward pass is two matrix multiplies, so you can count the multiplications exactly:

```
layer 1: W1 is (32 x 64) -> 32 x 64 = 2048 multiplies
layer 2: W2 is (10 x 32) -> 10 x 32 = 320 multiplies
TOTAL forward pass      = 2368 multiplies per prediction
parameters (numbers stored) = 2048 + 32 + 320 + 10 = 2410
```

```
# ai_9_measure.py -- the cost meter
def count_cost(W1, W2):
    mults = W1.size + W2.size
    params = W1.size + W2.size + b1.size + b2.size
    return mults, params
print(count_cost(W1, W2))      # (2368, 2410)
```

EFFICIENCY LENS

This is the exact bookkeeping used on giant models — theirs says "70 billion parameters, so many trillion multiplies per token"; yours says 2410 and 2368. Same math, different zeros. From now on, every model you ever build gets a cost line in your log next to its accuracy line. Quality per multiply is your research metric.

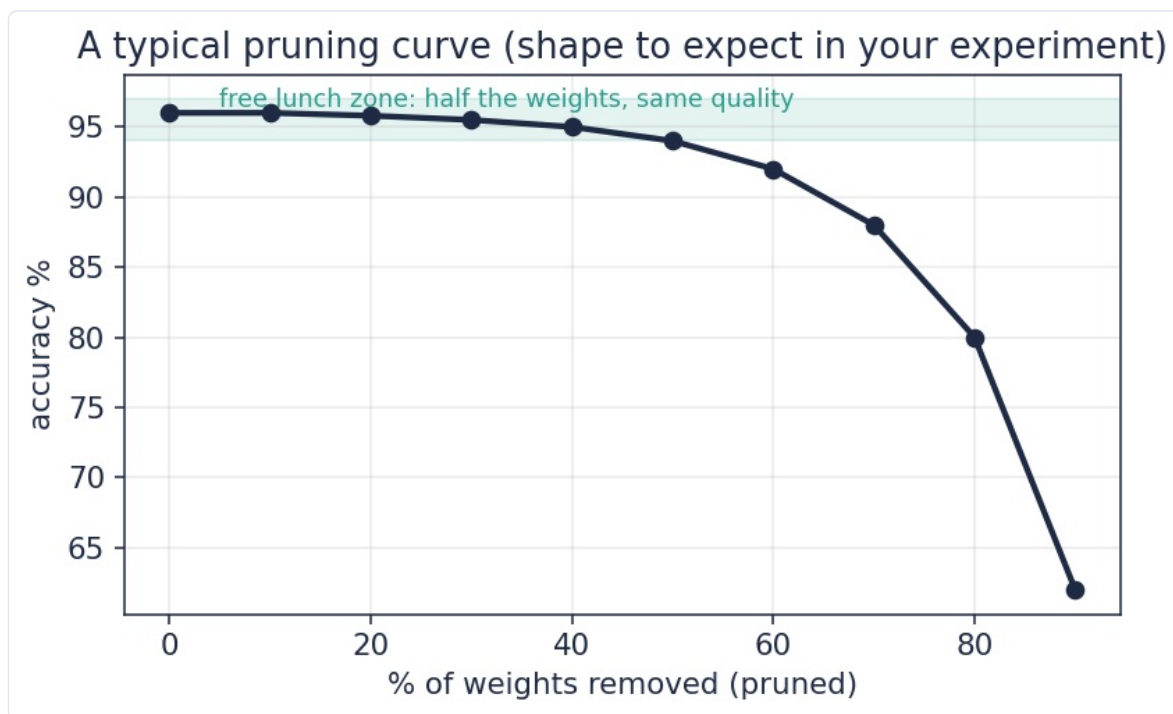
3.4 Efficiency Experiment #1 — Pruning

Hypothesis to test: many weights in a trained network are nearly useless — close to zero, contributing almost nothing. If true, we can delete them and keep the quality. Recipe:

1. Train your digit model fully. Record baseline test accuracy.
2. Find the smallest weights (by absolute size) and set them to exactly 0.
3. Re-measure test accuracy. Repeat at 10%, 30%, 50%, 70%, 90% pruned.

```
# prune the smallest fraction of W1 and W2
def prune(W, fraction):
    W = W.copy()
    cutoff = np.quantile(np.abs(W), fraction) # size below which we kill
    W[np.abs(W) < cutoff] = 0.0
    return W

for frac in [0.0, 0.1, 0.3, 0.5, 0.7, 0.9]:
    W1p, W2p = prune(W1, frac), prune(W2, frac)
    acc = test_accuracy(W1p, b1, W2p, b2) # your Week-2 grader
    print(f"pruned {int(frac*100):>2}% accuracy {acc:.1f}%")
```



The shape to expect: a long flat shelf, then a cliff. The shelf is the free lunch.

Typical result: **30-50% of weights can go with almost no accuracy loss.** Half the model was dead weight. You will have proven, with your own measurements, that trained networks are heavily over-provisioned — the founding observation of an entire efficiency subfield.

HONEST-EXPERIMENT HABIT

Prune using **ONLY** the weights' sizes — never peek at test accuracy to choose what to cut (that's tuning on the test set = leakage). And write your hypothesis + expected curve **BEFORE** running. Prediction first, result second: that's the difference between an experiment and a demo.

3.5 Efficiency Experiment #2 — Quantization (hand-rolled)

The idea: your weights are stored as ultra-precise numbers (~15 decimal digits). Does the network actually need that precision — or would a coarse approximation do? Test it by **rounding every weight to a fixed menu of allowed values**, like snapping to a grid:

```
4-bit example: only 16 allowed values, evenly spaced across the weight range
weights before: 0.7312, -0.2841, 1.1057, ...
weights after : 0.75, -0.30, 1.05, ... (each snapped to nearest allowed value)
```

```
def quantize(W, bits):
    levels = 2 ** bits                # 8 bits->256 values, 4->16, 2->4
    lo, hi = W.min(), W.max()
    step_size = (hi - lo) / (levels - 1)
    return lo + np.round((W - lo) / step_size) * step_size

for bits in [8, 4, 2]:
    acc = test_accuracy(quantize(W1,bits), b1, quantize(W2,bits), b2)
    print(f"{bits}-bit    accuracy {acc:.1f}%")
```

Typical result: **8-bit is free** (accuracy unchanged, storage cut $\sim 4\times$), **4-bit costs a little**, 2-bit hurts. Storage math for your model: 2410 numbers at 32 bits = ~ 9.6 KB; at 4 bits = ~ 1.2 KB. Same brain, one-eighth the size. This — literally this rounding trick, plus engineering — is how huge models get squeezed onto consumer laptops.

Then combine: prune 50%, THEN quantize to 4-bit. Do the savings stack? Measure and log it. (Usually largely yes — they attack different waste: pruning removes useless weights, quantization stores the useful ones coarsely.)

3.6 Write it up (your first research artifact)

One page, your own words, committed as `writeup.md`. Structure — the same one real papers use:

1. **Question:** how much can I shrink my digit model before quality drops?
2. **Setup:** model size, dataset, baseline accuracy, what you varied (one thing at a time).
3. **Results:** your pruning curve and quantization table. Numbers, not adjectives.
4. **Surprises & limits:** what you didn't expect; what this small test can't claim (one tiny dataset, one architecture — say so plainly).

Week 3 exercises

1. Prune 50% of W1 only, then 50% of W2 only. Which layer tolerates pruning better? Form a guess about why before measuring.
2. Quantize *and* keep training for 3 more passes (the nudges now fight the rounding). Does accuracy recover? (You've just invented the core idea of quantization-aware training.)
3. The "free lunch" isn't free storage-wise in our naive version — a pruned weight still occupies memory as a stored 0. What extra bookkeeping would let you truly NOT store the zeros? Sketch the idea in your log (no code needed).

ANSWERS

1: usually W1 tolerates more — it's bigger (2048 vs 320), so more redundancy; W2's few weights each carry more responsibility. **2:** yes, typically recovers most of the loss — training adapts around the coarse grid. **3:** store only the nonzero values plus their positions (a "sparse" format) — worth it once most weights are zero; that position bookkeeping is exactly why real sparse speedups need special formats/hardware.

WEEK 4 — Words In, Text Out

Goal: point the SAME machinery at language. By day 30 your network generates text — the seed of a language model — and you diagnose its limits like a researcher.

4.1 Text becomes numbers (and it's just inputs again)

Networks eat numbers. So give every character an ID number, and represent a character the same way we represented a digit label: **one-hot** — a vector of zeros with a single 1 at that character's position.

```
alphabet (say 30 chars): 'a'->0 'b'->1 ... 'h'->7 'i'->8 ... ' ' ->26 ...
"hi"   ->  h = [0,0,0,0,0,0,0,1,0,0,...]    (1 in position 7)
          i = [0,0,0,0,0,0,0,0,1,0,...]    (1 in position 8)
```

```
# ai_10_text.py -- text loader
text = open("my_research_log.txt").read().lower()
chars = sorted(set(text)) # your alphabet, from the data itself
char_to_id = {ch: i for i, ch in enumerate(chars)}
id_to_char = {i: ch for i, ch in enumerate(chars)}
V = len(chars)
print(f"{len(text)} characters, alphabet size {V}")

def one_hot_char(ch):
    v = np.zeros(V); v[char_to_id[ch]] = 1.0; return v
```

(Yes — train it on your own research log. Watching a model learn to imitate *your* writing makes every strength and failure vivid. Any text file of 10k+ characters works.)

4.2 The task: predict the next character

Here is the idea that powers every chatbot, in one line: **given the last few characters, guess the next one**. That's the entire training task. The dataset builds itself — slide a window along your text:

```
text: "the model learns"      window K = 4
input "the " -> label 'm'
input "he m" -> label 'o'
input "e mo" -> label 'd'
...thousands of examples, free, from any text.
```

Shape-wise, everything maps onto Week 2 exactly:

	Digits (Week 2)	Text (this week)
Input	64 pixels	K one-hot chars stuck end-to-end = K×V numbers
Output	10 scores (digits 0-9)	V scores (one per character)
Prediction	highest score	highest score = most likely next char

So `ai_10` is `ai_8` with different plumbing: input size K×V, output size V. The learning loop — unchanged, again.

4.3 Generation: the loop that writes

Once trained, make it write: give it K starting characters, predict the next, **append the prediction to the input**, slide the window, predict again. The model eats its own output — that feedback loop IS text generation.

```
def generate(seed, length=200):
    out = seed
    for _ in range(length):
        window = out[-K:] # last K chars
        x = np.concatenate([one_hot_char(c) for c in window])
        scores = W2 @ squash(W1 @ x + b1) + b2

        # temperature sampling -- see note
        p = np.exp(scores / 0.8)
        p = p / p.sum()
        next_id = np.random.choice(V, p=p)
        out += id_to_char[next_id]
    return out

print(generate("the "))
```

Why sample instead of always taking the top score: always-the-top gets stuck in ruts ("the the the the"). Instead, convert scores to probabilities and roll a weighted die. The 0.8 is the **temperature**: lower = safer and more repetitive, higher = wilder and less coherent. Play with it — it's one number and the personality change is dramatic.

What to expect (calibrate your hopes): after training, output like *"the modet learus the wingt of the smoll..."* — wordish nonsense with real short words and plausible letter-patterns. For a K-character window with no memory beyond it, that IS success. It has learned the *texture* of your language: which letters follow which. What it lacks is long-range memory — and feeling that ceiling yourself is the entire setup for attention and the tiny GPT next month.

4.4 Diagnose like a researcher

Don't just say "it's bad." Rank the suspects, then test the top one — one dial at a time:

Hypothesis	Test	Expected if true
H1: window too short (K)	K = 3 vs 8 vs 16	longer K -> fewer mid-word breaks, more word-like output
H2: model too small (H)	H = 32 vs 128	bigger H -> lower loss, better texture (and more compute — log it)
H3: too little data	train on 5k vs all chars	more data -> less parroting of specific memorized phrases

Measure with both numbers (training loss, and accuracy of next-char prediction on held-out text — yes, split text into train/test too!) and eyes (read the samples; save them in the log — they're your favorite artifacts later).

4.5 The efficiency pass (reuse pays off)

Your Week-3 scripts don't care what the model does — weights are weights. Point `prune()` and `quantize()` at your BEST text model and produce the same curves. Then write the one-line summary that is, in miniature, the thesis of your whole research direction: *"my text model at 4-bit and 50% pruned: __% smaller, quality change: __."*

EFFICIENCY LENS

Notice what just happened: you built the measuring tools ONCE and they transferred to a completely different task unchanged. That's why we keep one codebase. Tools compound; throwaway scripts don't.

4.6 Day 30: the capstone README

Turn the repo into a story anyone can read. README sections: **what I built** (from-scratch net → digits → text), **what I measured** (accuracy tables, pruning/quantization curves), **favorite generated samples** (include the funny ones), **what I learned** (5 honest bullets, including what confused you), **what's next** (embeddings → attention → tiny GPT → distillation). Tag it `v0.3`. That page is a portfolio piece almost no self-taught learner can show: not "I did a course" but "I built, measured, and shrank an AI from scratch."

Week 4 exercises

1. Train the same model on a DIFFERENT text (a public-domain book). Compare generated samples side by side. What did each model's "texture" pick up from its diet?
2. Set temperature to 0.1, then 2.0. Paste both outputs in your log and explain the difference using the weighted-die picture.
3. Estimate your text model's multiplies-per-character with `ai_9`. How many multiplies to generate a 200-char sample? (Frontier chatbots do many billions per character. Write down the ratio — and keep it: it's your first feel for the scale gap your research lives in.)

ANSWERS

1: vocabulary and rhythm shift with the diet — model output is a mirror of training data (the deepest lesson in modern AI, seen firsthand). **2:** 0.1 = repetitive loops of safest patterns; 2.0 = near-random gibberish; the die's weights get sharpened or flattened. **3:** $\text{mults} = H \times (K \times V) + V \times H$ per char; $\times 200$ for the sample. Typical answer: a few million — versus trillions. That gap, and closing it from both ends, is the field you're entering.

After Day 30 — where this goes

You will be standing at the exact doorway of Stage 4. The path continues: **embeddings** (characters/words become learned vectors instead of one-hots — your first big quality jump), **attention** (the fix for the short-window ceiling you just felt), a **tiny GPT** assembled from parts you already own, and then the experiment your whole track points at: **distillation** — your small model learning from a big teacher, KL divergence and all.

The four sentences to carry forward

1. A model is scoring tables stacked with bends between them — and you can trace every number in yours by hand.
2. Learning is guess → measure wrongness → pass blame backward → nudge — nothing more, at any scale.
3. The honest grade is always on data the model never saw; when a result looks amazing, hunt for the leak first.
4. Quality per multiply is your metric; one dial at a time, hypothesis before result, is your method.

- ☐ Week 1: my 2-layer network learns XOR and I can trace the blame on paper
- ☐ Week 2: it reads handwritten digits at 90%+ on data it never saw
- ☐ Week 3: I shrank it, measured exactly what that cost, and wrote it up
- ☐ Week 4: it generates text, I diagnosed its limits, and the repo tells the story